04Lab
Verified Specification, ADTs, type classes, and monads

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

One slide sum-up on Advanced Testing

Software models and programming languages

- software models are specification of a software systems, useful for design and verification
- software models capturing system behaviour are systems per se, to be properly engineered
- programming languages (like Scala) are very good at programming software models

An example formal model: ADTs

- type, constructors, operators, and axioms (or other math)
- can be turned into Scala ADTs, with trait/opaque types, and using ScalaCheck/Test

From ADTs to type-classes and monads

- type-classes can be used to factorise empowerment of existing, simple ADTs
- an example is monads, a flexible way to compose functional abstractions
- among the many applications, one allows you to write an entire MVC application

Starting point and goals

References

- 04-repo: `https://github.com/mviroli/asmd24-public-04-advanced-programming`
- ScalaTest: `https://www.scalatest.org/user_guide`
- ScalaCheck: `https://scalacheck.org/documentation.html`
- ADT: `https://softwarefoundations.cis.upenn.edu/vfa-current/ADT.html`

General goals

- be operative with ScalaCheck/Test
- play with ADTs and ScalaCheck
- play with monads
- engineer with monads

Operational Task

Step 1 – Download and check

- Download the lab repository: `https://github.com/mviroli/asmd24-public-04-advanced-programming`
- Import it in IntelliJ as an SBT project
- Check that all Scala check tests (`SequenceCheck`) run correctly
- Note!! The rest of the code should be implemented inside/using the package 'scala.lab04'

Step 2 – Play with ScalaCheck

- Check the ScalaCheck documentation
- Examine the existing ScalaCheck tests for sequences in `test/scala/lab04`
- Note that currently only `map` operation is checked, while `sum` and `filter` are just implemented
- Complete the tests by implementing ScalaCheck properties for:
  - `sum` operation (verify it behaves correctly)
  - `filter` operation (verify it satisfies expected properties)
- Try to implement a new operation like `flatMap` and write ScalaCheck tests for it
- Explore ScalaCheck parameters:
  - How many tests are generated by default?
  - How to change the number of tests?
  - How to modify the random seed?
  - Examine other available parameters
- Compare ScalaCheck with ScalaTest:
  - Can ScalaTest perform parameterized tests?
  - What are the key differences between the two testing frameworks?

Tasks

ADT-VERIFIER

- Define a formal Abstract Data Type (ADT) for sets with essential operations: `union`, `intersection`, `contains`
- Examine the current implementation in `scala.lab04.SetADT` and understand its structure
- Complete the property-based tests in `SetADTCheck` by:
  - Adding missing algebraic properties (commutativity, associativity, idempotence) for `union` and `intersection` (e.g., $A \cup B = B \cup A$)
  - Implementing cross-property tests (e.g., relationship between union and intersection)
  - Ensuring properties reflect the mathematical axioms of sets
- Once tests are complete, implement an alternative version of SetADT using a Tree-based structure
- Adapt the tests minimally to work with your new implementation

JAVA-SCALA-CHECK

- Use ScalaCheck as a property-based testing tool for Java code (leveraging Scala's Java interoperability)
- Implement the following steps:
  - Create a simple immutable Java class (e.g., `Point2D` with `x`, `y` coordinates and methods like `distanceTo`, `translate`, `rotate`)
  - Write a comprehensive ScalaCheck test suite that verifies mathematical properties such as: Distance symmetry (i.e., `a.distanceTo(b) == b.distanceTo(a)`), Triangle inequality, Rotation invariants
  - Extend to testing Java standard library classes like `java.util.ArrayList` or `java.util.TreeSet`
- Reflect on the advantages and limitations of using ScalaCheck as a cross-language testing DSL

MONAD-VERIFIER

Define ScalaCheck properties for Monad axioms, and prove that some of the monads given during the lesson actually satisfy them. Derive a general methodology to structure those tests.

MVC-ENGINEER

Start with the given MVC monadic application: extend it to realise a more complex application, e.g. the DrawNumberGame. Of course, be fully monadic. Up to which complexity can one reach? Could we come up with a simple MVC application with reactive GUI, and/or could a GameLoop be framed into a fully monadic application?

ADVANCED-FP-LLM

LLMs/ChatGPT can arguably help in write/improve/complete/implement/reverse-engineer ADT specifications, ADTs in Scala, and monadic specifications. Check if/whether this is the case.